

Architectures and Platforms for Artificial Intelligence - Module 1

Parallel Lloyd's K-means algorithm

Lorenzo Pellegrino, lorenzo.pellegrino2@studio.unibo.it

Student ID: 0001189550

1 Introduction

In this report, we investigate the scalability improvements achievable through two parallelization paradigms applied to Lloyd's K-means algorithm: OpenMP for multi-core CPU parallelism and CUDA for GPU acceleration.

1.1 Experimental Setup

Since the primary focus of this work is on scalability and parallel performance rather than solution quality, the convergence tolerance criterion has been deliberately omitted from all implementations, fixing the number of iterations at $I = 100$. This ensures a fair and consistent comparison of execution times across configurations, decoupled from the variability introduced by early stopping.

To generate the workloads used for testing, the provided `inputgen.c` utility was used. Finally all experiments were conducted on the university HPC cluster using the 140 partition, since it provided the best setup for testing both the implementations and also to ensure that they are both evaluated under comparable hardware conditions.

1.2 General Approach

The parallelization strategy was driven by a preliminary analysis of the algorithm's two main phases: `classify()` and `update_centroids()`.

Considering that the iterations are fixed, given N data points, D dimensions, and K clusters, the former dominates the computational load with $O(N \times D \times K)$ per iteration, while the latter accounts for only $O(N \times D)$. As K grows the latter becomes increasingly negligible, so the asymptotic complexity of the full iteration is determined by `classify()` alone. Their execution times are therefore measured separately throughout all experiments, in addition to the total, to better track relative improvements to each phase.

Since $N \gg K$ by definition and $N \gg D$ in practice, the focus was placed on the outermost loop over N . It exhibits no cross-iteration dependencies in both functions, since each point is processed independently. This makes it embarrassingly parallel and therefore the primary target for achieving optimal performances in both the implementations.

This approach introduces shared write conflicts: `classify()` accumulates into `counts[]`, a vector of size K , and `update_centroids()` accumulates into `new_centroids[]`, an array of size $K \times D$. The contention rates are similar since both grow with N and are inversely proportional to K , the only difference is that the latter accumulates also over D dimensions so it may be more sensitive to that higher work. However we can resolve these conflicts via reduction. Each parallel worker maintains a private copy of the contended structure, accumulates independently, and merges at the end of the parallel region. The specific implementations are discussed separately.

2 OpenMP

2.1 Approach

We parallelize the N dimension across P processors, resulting in a workload of $O(\frac{N}{P} \times D \times K)$ per processor. Since each iteration performs a constant amount of work, `schedule(static)` is the natural choice. For these reasons, we expect a near-linear speedup. About the reduction strategy, OpenMP automatically handles it with `reduction()`.

2.2 Experiments structure

The experimental grid is defined as follows:

- $N \in \{50k, 100k, 150k, \dots, 400k\}$, chosen as multiples of $N_0 = 50k$ to support weak scaling.
- $P \in \{1, 2, \dots, 8\}$, using all CPUs available on the partition. Simultaneous multi-threading was not available.
- $D \in \{20, 60, 100\}$ and $K \in \{3, 5, 10\}$, covering a range of low-dimensional configurations.

N is varied as the primary scaling parameter. Although D and K are not dominant factors in computational complexity, their effect on parallelization efficiency was not assumed negligible a priori. When P increases, each thread operates on a smaller portion of **data**, which may fit in L3 cache rather than requiring RAM accesses and reducing cache misses, causing additional speedups. The grid over (D, K) was therefore included to detect any such behavior before fixing these parameters for subsequent experiments.

Each configuration experiment was repeated five times to reduce measurement noise. The reported values are means with propagated standard deviations.

2.3 Results

Due to space requirements and readability, we do not show the results of the whole grid for each metric but show only representative results.

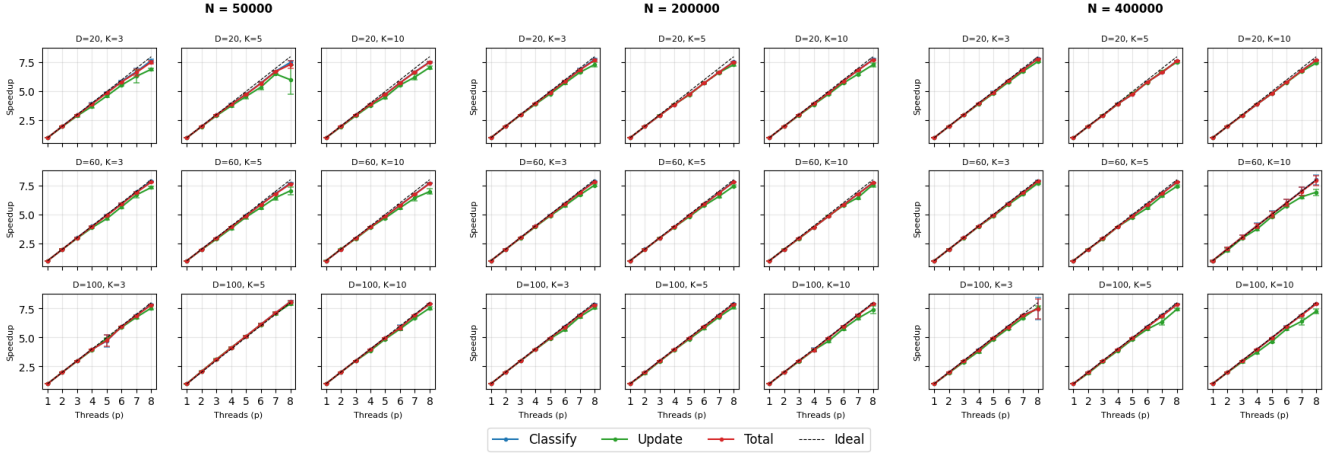


Figure 1: Speedup $S(p, N, D, K) = \frac{T_{\text{parallel}}(1, N, D, K)}{T_{\text{parallel}}(p, N, D, K)}$

As shown in figure 1, the speedups of all functions approach the ideal linear reference, consistent with the embarrassingly parallel structure of the loop over N . The curves, when they are well defined by low standard deviations, do not change across all nine (D, K) combinations at each fixed N , confirming that these parameters do not influence parallelization efficiency within the tested ranges. D and K are therefore fixed in the next metrics to make their representation easier.

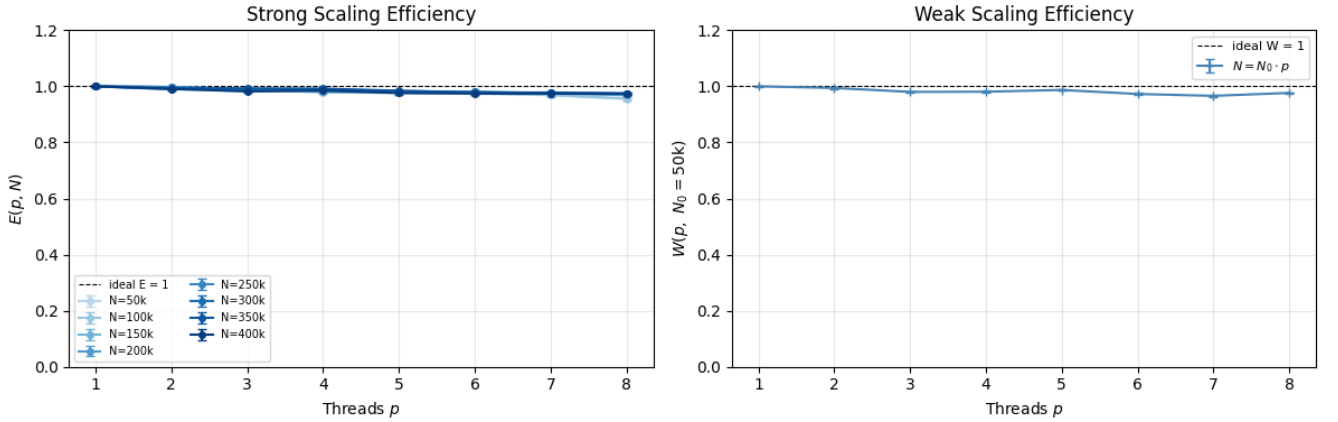


Figure 2: Strong scaling efficiency $E(p, N) = \frac{S(p, N)}{p}$ and weak scaling efficiency $W(p, N_0) = \frac{T_{\text{parallel}}(1, N_0)}{T_{\text{parallel}}(p, p \cdot N_0)}$ for $D = 60$ and $K = 5$.

To further evaluate parallelization quality, strong and weak scaling efficiency are reported in figure 2. For weak

scaling, since the parallelization is over N only, the problem size is scaled as $N = P \cdot N_0$ with $N_0 = 50k$, so that each thread always processes the same number of points regardless of P .

We can observe that both the metrics show the same behavior, staying consistently around their ideal value 1.

A high value of the strong scaling metric means that the program scales very well when adding cores, and that the overheads introduced by them are really small, as if they were working independently.

A high value of the weak scaling metric means that the program scales well when both the problem size and the number of cores increase. This indicates that the overheads associated with larger problem sizes and additional cores are minimal, allowing the system to handle the increased workload efficiently.

2.4 Optimization impact

During the initial exploration phase, `-O3` and explicit SIMD vectorization via pragma directives were used to make the experimental grid feasible, since the unoptimized baseline was too slow for exhaustive sweeps. SIMD was particularly interesting to evaluate on the inner loops over D , which operate on contiguous arrays with stride-1 access and no cross-iteration dependencies. Since SIMD directives require compiler optimization to take effect, the two flags were tested together. The results reported above refer to the final unoptimized version while here we highlight the interesting behavior observed under this optimized configuration. The optimizations yield a speedup exceeding $10\times$ relative to `-O0`, with `-O3` accounting for the majority of the gain and SIMD providing an incremental improvement that scales with D , as shown in figure 3.

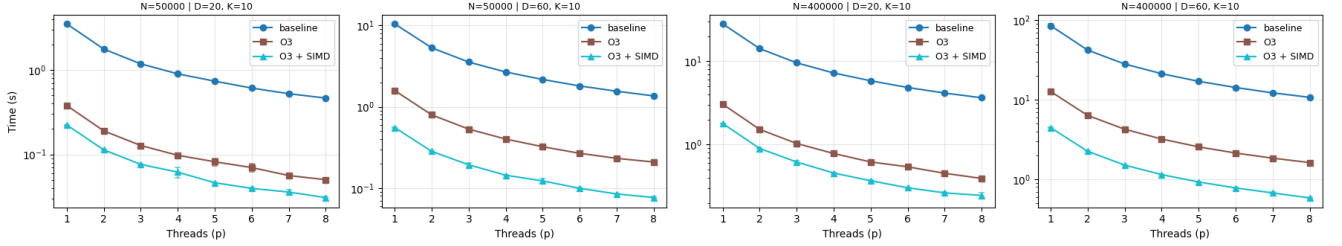


Figure 3: Execution times under different compilation stages for representative (N, D, K) configurations.

Since optimizations operate independently within each core, parallelization metrics would be expected to remain unaffected. However, `-O3` substantially reduces the computation time of `update_centroids()`, shrinking the N/P work term to the point where the non-deterministic overhead of the parallel reduction becomes a significant fraction of the total. This exposes an underlying run-to-run variability that propagates into ratio-derived metrics: the resulting standard deviations on the speedup occasionally produce apparent superlinear values, as visible in figure 4, and degrades the efficiencies, as shown in figure 5. The effect is more pronounced at small N and higher thread counts, where N/P is smallest and the reduction overhead weighs most. The total speedup and efficiency are less affected since `update_centroids()` accounts for a small fraction of the overall runtime, which is dominated by `classify()`.

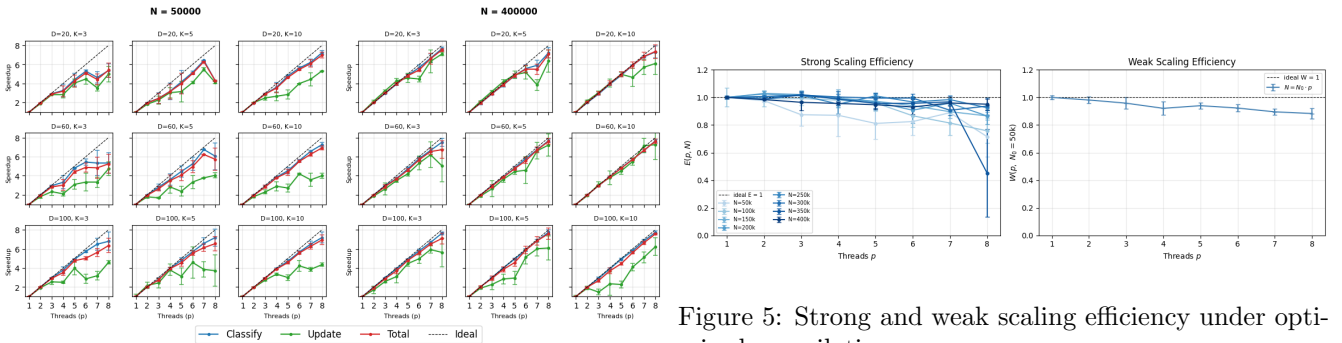


Figure 5: Strong and weak scaling efficiency under optimized compilation.

Figure 4: Speedup under optimized compilation.

The total speedup remains linear and efficiencies stay above 0.8, while the walltime is strongly reduced. Nevertheless, the unoptimized baseline is better evaluated from a scaling point of view.

3 CUDA

3.1 Approach

Following the general approach, each CUDA thread processes one data point. Regarding the reduction strategy to solve write conflicts:

1. Atomic access on shared memory

Each block allocates a private copy of the contended structure in shared memory and initialises it in parallel using the block’s own threads B . Each thread then accumulates into it via `atomicAdd` on shared memory, whose latency is far lower than that of global atomics. After a barrier, each slot is flushed to global memory with a single `atomicAdd`, reducing total global contention. This approach presents some limitations based on the problem dimensions. First, contention per slot grows as $K \times D$ (or K) shrinks relative to B : when all B threads in a block write to only a few shared slots, serialization at each address increases. Second, as $K \times D$ grows the buffer itself consumes more shared memory, reducing the number of blocks that can reside simultaneously on an SM and therefore lowering occupancy¹.

2. Binary-halving reduction tree

Intra-block atomics are eliminated entirely. Instead, shared memory is used as an accumulator buffer and progressively reduced via binary halving, then after a barrier only one thread issues one global `atomicAdd` per slot. Since the kernels have different structures and memory footprints, they required different implementations:

- In `classify()`: a $K \times B$ integer buffer is allocated. Each thread writes a 0/1 indicator for all K clusters simultaneously, so all K prefix sums are reduced in a single pass.
- In `update.centroids()`: in order to reduce the potential number of barriers and their overhead, the buffer was widened from B to $D \times B$ to process all D dimensions of a cluster simultaneously in a single pass. Then the problem was that the size could exceed the GPU hardware limit, so tiling over D was introduced to fit the shared memory per block.

This approach assigns each thread a private slot with no contention at all, paying the cost in $\log_2 B$ synchronization steps instead. Its shared-memory footprint for the update kernel is bounded by the device limit by construction via tiling.

In both strategies and for both functions, B is not fixed manually in order to avoid choosing an inefficient value or enlarging the experiments iterating over representative values, but it is selected per-kernel via a specific CUDA API². It maximizes the theoretical occupancy and it generally can improve parallel execution and hide memory latency. The tree-halving reduction strategy requires the block dimension to be a power of two but the API couldn’t ensure this, so in that case it is approximated with the nearest power of two.

3.2 Experimental setup

The experimental grid is the same as for OpenMP, to allow a speedup comparison between implementations:

- $N \in \{50k, 100k, 150k, \dots, 400k\}$.
- $D \in \{20, 60, 100\}$ and $K \in \{3, 5, 10\}$.

Moreover, since the innermost access dimension is D and since it has independence properties it could benefit from coalesced access. In the default AoS (Array of Structures) layout, `data[]` is stored as an $N \times D$ row-major matrix: consecutive threads in a warp access addresses separated by D floats, producing non-coalesced transactions and multiplying effective memory traffic by up to D . For this reason, in addition to AoS, the SoA (Structure of Arrays) layout was also employed. `data[]` is transposed once on the host to a $D \times N$ layout before the GPU phase, so that consecutive threads read consecutive addresses and full coalescing is achieved. `centroids[]` is instead kept in AoS layout even in the SoA variant. In the inner loop of `sqdist.d`, all threads in a warp iterate over the same centroid

¹The ratio of the number of active warps to the maximum number of warps that can be active on a given SM at any one time.

²`cudaOccupancyMaxPotentialBlockSize`

and the same dimension at each step, so every thread reads the identical address and transposing centroids to SoA would therefore bring no memory-bandwidth benefit.

As in the OpenMP experiments, each configuration was repeated five times. The reported values are means with propagated standard deviation.

3.3 Results

The metric to evaluate the CUDA implementations is the throughput, the number of operations per second. In our case the total throughput was computed as $T = \frac{N \times D \times K \times I}{\text{Walltime}}$ ops/s, following the total asymptotic computational cost.

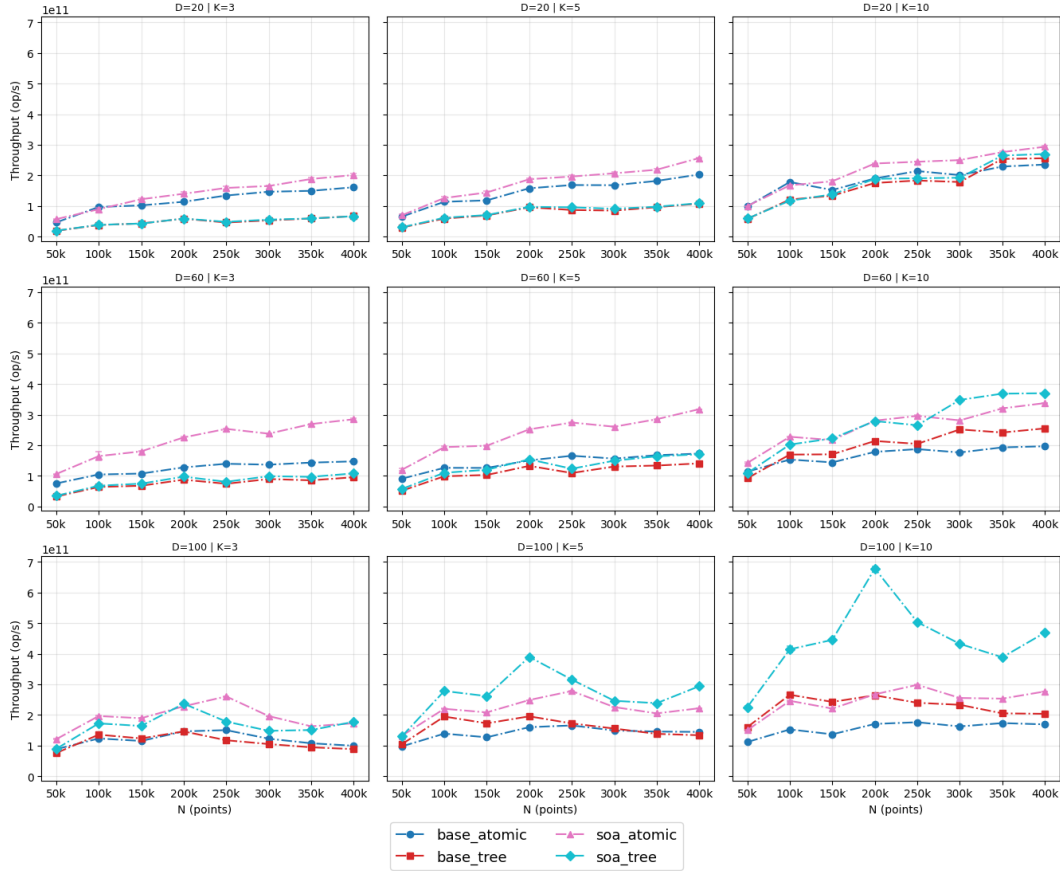


Figure 6: Throughput for (N, D, K) configurations and different implementations.

From the results, even if the changes across plots are not so clear, we can make some observations:

- **All variants scale with N :** throughput is roughly flat or gently increasing across the N axis for every configuration. This is expected since the GPU hides memory latency through massive parallelism, so larger N simply improves hardware utilization until bandwidth is saturated.
- **AoS layouts suffer from uncoalesced memory access:** consecutive threads in a warp read elements that are D floats apart under AoS, so each memory transaction fetches a full cache line but delivers only one useful float per thread, wasting bandwidth proportionally³ to D . Both SoA variants therefore consistently outperform their AoS counterparts, and the gap widens as D grows.

³Actually, if $1 < D < \text{dim.cache_line}$ we still have those gaps but we are still able to take multiple values from the same fetched cache line, but the long-term effect is the one described. In particular on the L40 the cache line dimension is 64 bytes so 16 values, but we are always over it in our ranges.

- **The `base_tree` is more robust to D and K than `base_atomic`:** the tiled update kernel caps its shared-memory usage following the device shared-memory limit regardless of K or D . It therefore maintains stable occupancy at larger configurations.
- **The `soa_atomic` is the best performer for small and medium D :** coalesced memory access combines with the low synchronization overhead of shared-memory atomics, which flush to global memory with only one `atomicAdd` per slot per block. For small D and K the $K \times D$ shared footprint is negligible, and this variant leads all others.
- **The `soa_tree` dominates at large D and K :** when D and K are large, the atomic update kernel accumulates three compounding disadvantages which are the rising shared-memory pressure, the reduced occupancy, and the growing contention on the $K \times D$ shared slots. The tree reduction eliminates all three: tiling keeps shared memory bounded, each thread writes to a private slot, and no shared-memory atomics are used at all.
- **The atomic/tree crossover tracks the $K \times D$ product:** for low $K \times D$ the atomic approach wins due to lower synchronization overhead. As $K \times D$ grows past a certain threshold the occupancy drop and atomic contention of the atomic variants move the balance in favor of the tree reduction. This crossover is visible in the transition from the top-left panels to the bottom-right panel.

4 Comparison

Another important evaluation metric is the speedup between OpenMP (with $P = 8$) and CUDA implementations.

Config (N,D,K)	Baseline	O3+SIMD	AoS+Atomic	AoS+Tree	SoA+Atomic	SoA+Tree
(50k, 20, 5)	265.60 ms	8.62×	16.11×	34.91×	16.54×	37.29×
(50k, 60, 5)	736.80 ms	13.02×	25.28×	44.67×	28.20×	58.87×
(50k, 100, 5)	1200.00 ms	12.55×	50.21×	46.83×	62.45×	63.44×
(200k, 20, 5)	1038.20 ms	14.58×	49.97×	82.20×	50.72×	97.49×
(200k, 60, 5)	2899.80 ms	14.50×	64.12×	72.80×	74.42×	121.73×
(200k, 100, 5)	4789.40 ms	13.67×	93.72×	76.76×	186.58×	119.20×
(400k, 20, 5)	2043.60 ms	13.44×	55.33×	103.90×	55.98×	131.31×
(400k, 60, 5)	5762.40 ms	15.72×	67.62×	82.95×	81.83×	152.56×
(400k, 100, 5)	9568.40 ms	13.28×	64.11×	69.32×	140.62×	106.23×
Mean+Std	—	13.26 ± 1.99	54.05 ± 23.08	68.26 ± 22.09	77.48 ± 54.16	98.68 ± 38.04

Table 1: Speedup of all implementations relatively to the OpenMP baseline version. The baseline is reported with walltime (ms) to give an absolute reference while the others are reported as speedups. The last row is the average speedup across configurations.

5 Conclusion

The OpenMP implementation achieves near-ideal parallelization efficiency and scaling across all tested configurations. Compiler optimizations yield substantial walltime reductions, with a speedup over the unoptimized baseline that is remarkably invariant across configurations, as confirmed by the low standard deviation in the table 1.

The CUDA implementations outperform the unoptimized OpenMP baseline by a wide margin, but since `nvcc` compiles device code at `-O3` by default, the more meaningful comparison is against the `O3+SIMD` OpenMP version, which reduces the apparent gap considerably. The speedup grows with N and is strongly configuration-dependent across CUDA variants, following the $K \times D$ product.

Alternative designs were explored during development but did not yield improvements. Direct global atomics performed worse due to the high latency. Fusing the `update_centroids` computation into the `classify` kernel to amortize kernel launch overhead produced no measurable benefit, confirming that kernel launch overhead is negligible relative to the cost of memory access and reduction contention, which represent the true bottleneck.